C++ Programming: From Problem Analysis to Program Design, Third Edition

Chapter 7: User-Defined Functions II

Objectives

In this chapter you will:

- Learn how to construct and use void functions in a program
- Discover the difference between value and reference parameters
- Explore reference parameters and value-returning functions
- Learn about the scope of an identifier

Objectives (continued)

- Examine the difference between local and global identifiers
- Discover static variables

Void Functions

- Void functions and value-returning functions have similar structures
  - Both have a heading part and a statement part
- User-defined void functions can be placed either before or after the function `main`

Void Functions (Continued)

- The program execution always begins with the first statement in the function `main`
- If user-defined void functions are placed after the function `main`
  - The function prototype must be placed before the function `main`

Void Functions (continued)

- A void function does not have a return type
- The `return` statement without any value is typically used to exit the function early
- Formal parameters are optional
- A call to a void function is a stand-alone statement

`void` Functions Without Parameters

- Function definition syntax:

```
void functionName()
{
    statements
}
```

- `void` is a reserved word
- Function call syntax:

```
functionName();
```

void Functions With Parameters

- Function definition syntax:

```
void functionName(formal parameter list)
{
    statements
}
```

FORMAL PARAMETER LIST

The formal parameter list has the following syntax:

```
dataType& variable, dataType& variable, ...
```

FORMAL PARAMETER LIST

The formal parameter list has the following syntax:

```
dataType& variable, dataType& variable, ...
```

FUNCTION CALL

The function call has the following syntax:

```
functionName(actual parameter list);
```

ACTUAL PARAMETER LIST

The actual parameter list has the following syntax:

```
expression or variable,expression or variable, ...
```

EXAMPLE 7-2

```
void funexp(int a, double b, char c, int x)
{
    .
    .
    .
}
```

The function `funexp` has four parameters.

EXAMPLE 7-3

```
void expfun(int one, int& two, char three, double& four)
{
    .
    .
    .
}
```

The function `expfun` has four parameters: (1) `one`, a value parameter of type `int`; (2) `two`, a reference parameter of type `int`; (3) `three`, a value parameter of type `char`, and (4) `four`, a reference parameter of type `double`.

`void` Functions With Parameters (continued)

- A formal parameter receives a copy of the content of corresponding actual parameter

- <u>Reference Parameter</u> - a formal parameter that receives the location (memory address) of the corresponding actual parameter

Value Parameters

- If a formal parameter is a value parameter
  - The value of the corresponding actual parameter is copied into it
- The value parameter has its own copy of the data
- During program execution
  - The value parameter manipulates the data stored in its own memory space

Reference Variables as Parameters

- If a formal parameter is a reference parameter
  - It receives the address of the corresponding actual parameter
- A reference parameter stores the address of the corresponding actual parameter

Reference Variables as Parameters (continued)

- During program execution to manipulate the data
  - The address stored in the reference parameter directs it to the memory space of the corresponding actual parameter

Reference Variables as Parameters (continued)

- A reference parameter receives the address of the actual parameter
- Reference parameters can:
  - Pass one or more values from a function
  - Change the value of the actual parameter

Reference Variables as Parameters (continued)

- Reference parameters are useful in three situations:
  - Returning more than one value
  - Changing the actual parameter
  - When passing the address would save memory space and time

Parameters & Memory Allocation

- When a function is called
  - Memory for its formal parameters and variables declared in the body of the function (called local variables) is allocated in the function data area
- In the case of a value parameter
  - The value of the actual parameter is copied into the memory cell of its corresponding formal parameter

Parameters & Memory Allocation (continued)

- In the case of a reference parameter
  - The address of the actual parameter passes to the formal parameter
- Content of the formal parameter is an address

Parameters & Memory Allocation (continued)

- During execution, changes made by the formal parameter permanently change the value of the actual parameter

EXAMPLE 7-7

The following program shows how reference and value parameters work.

```
//Example 7-7: Reference and value parameters

#include <iostream>

using namespace std;

void funOne(int a, int& b, char v);
void funTwo(int& x, int y, char& w);

int main()
{
    int num1, num2;
    char ch;

    num1 = 10;                                          //Line 1
    num2 = 15;                                          //Line 2
    ch = 'A';                                           //Line 3
```

```
    cout << "Line 4: Inside main: num1 = " << num1
         << ", num2 = " << num2 << ", and ch = "
         << ch << endl;                                  //Line 4

    funOne(num1, num2, ch);                              //Line 5

    cout << "Line 6: After funOne: num1 = " << num1
         << ", num2 = " << num2 << ", and ch = "
         << ch << endl;                                  //Line 6

    funTwo(num2, 25, ch);                                //Line 7

    cout << "Line 8: After funTwo: num1 = " << num1
         << ", num2 = " << num2 << ", and ch = "
         << ch << endl;                                  //Line 8

    return 0;
}
```

```
void funOne(int a, int& b, char v)
{
    int one;

    one = a;                                                    //Line 9
    a++;                                                        //Line 10
    b = b * 2;                                                  //Line 11
    v = 'B';                                                    //Line 12

    cout << "Line 13: Inside funOne: a = " << a
         << ", b = " << b << ", v = " << v
         << ", and one = " << one << endl;                      //Line 13
}

void funTwo(int& x, int y, char& w)
{
    x++;                                                        //Line 14
    y = y * 2;                                                  //Line 15
    w = 'G';                                                    //Line 16

    cout << "Line 17: Inside funTwo: x = " << x
         << ", y = " << y << ", and w = " << w
         << endl;                                               //Line 17
}
```

Sample Run:

```
Line 4: Inside main: num1 = 10, num2 = 15, and ch = A
Line 13: Inside funOne: a = 11, b = 30, v = B, and one = 10
Line 6: After funOne: num1 = 10, num2 = 30, and ch = A
Line 17: Inside funTwo: x = 31, y = 50, and w = G
Line 8: After funTwo: num1 = 10, num2 = 31, and ch = G
```

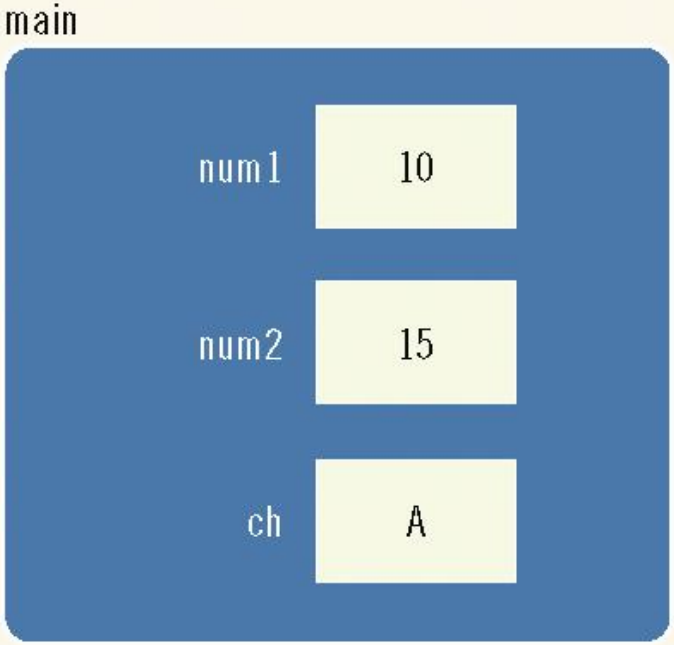

FIGURE 7-5 Values of the variables after the statement in Line 3 executes

```
one = a;                                    //Line 9
```

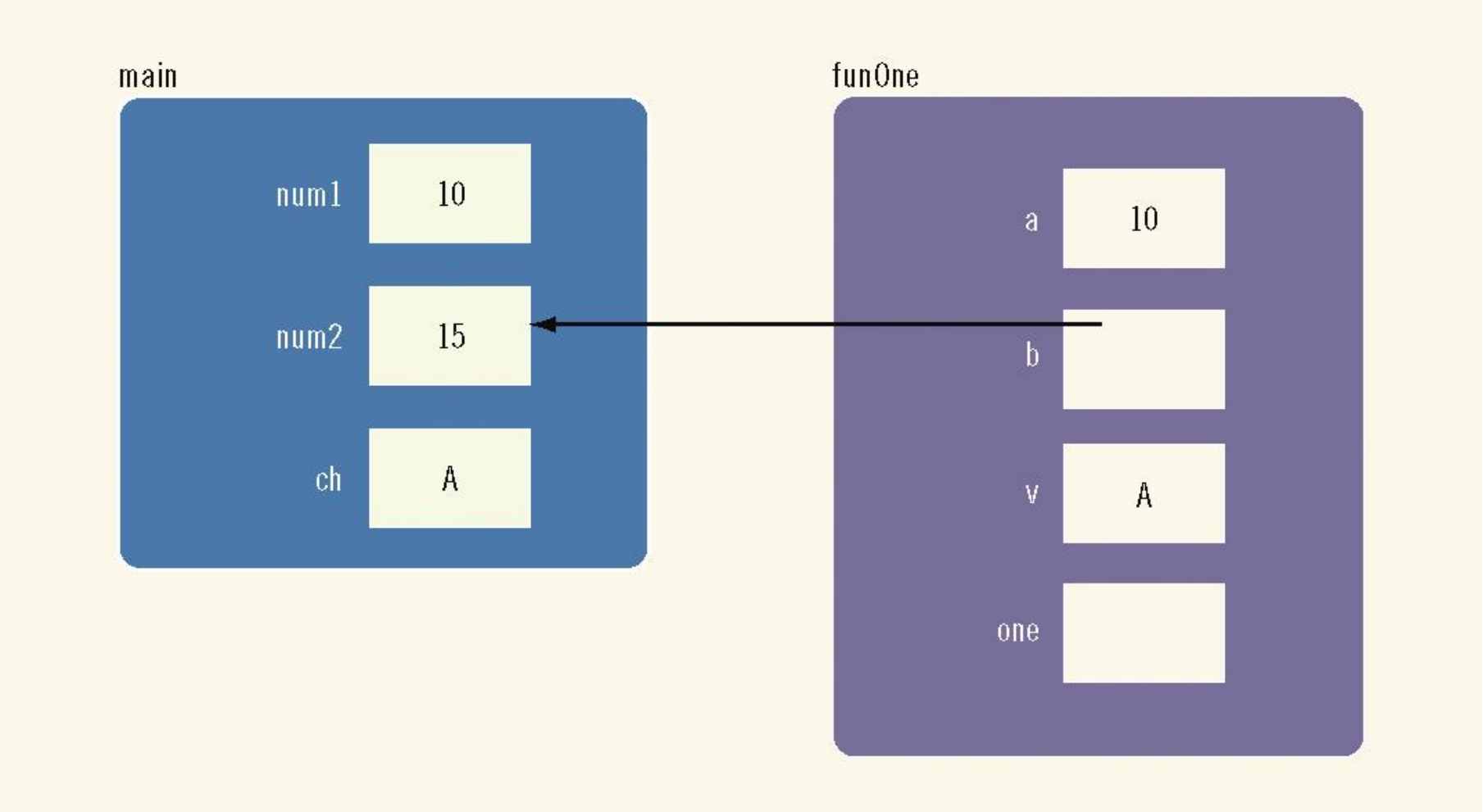

FIGURE 7-6 Values of the variables just before the statement in Line 9 executes

```
one = a;                                                    //Line 9
```

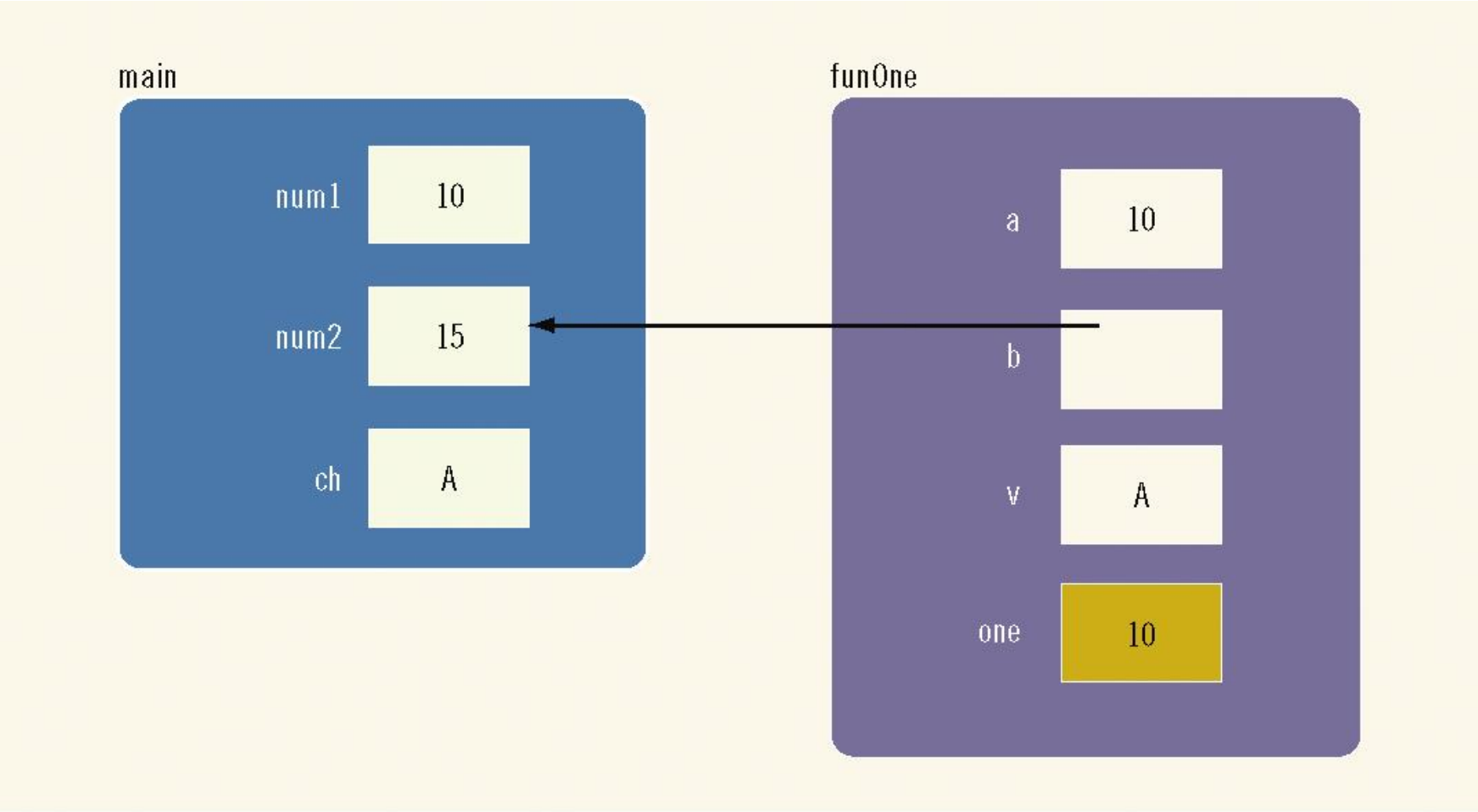

FIGURE 7-7 Values of the variables after the statement in Line 9 executes

```
a++;                                                    //Line 10
```

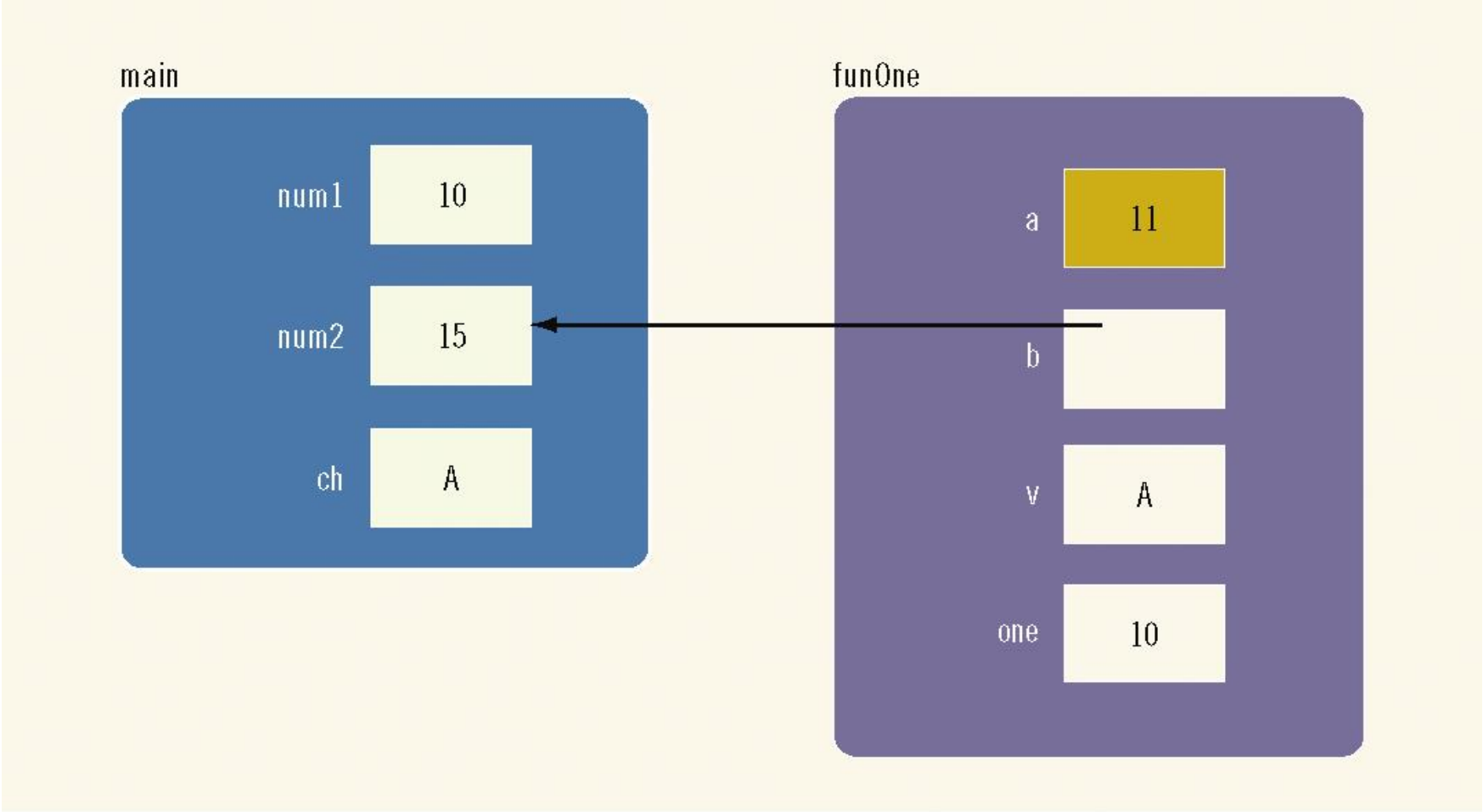

FIGURE 7-8 Values of the variables after the statement in Line 10 executes

```
b = b * 2;                                    //Line 11
```

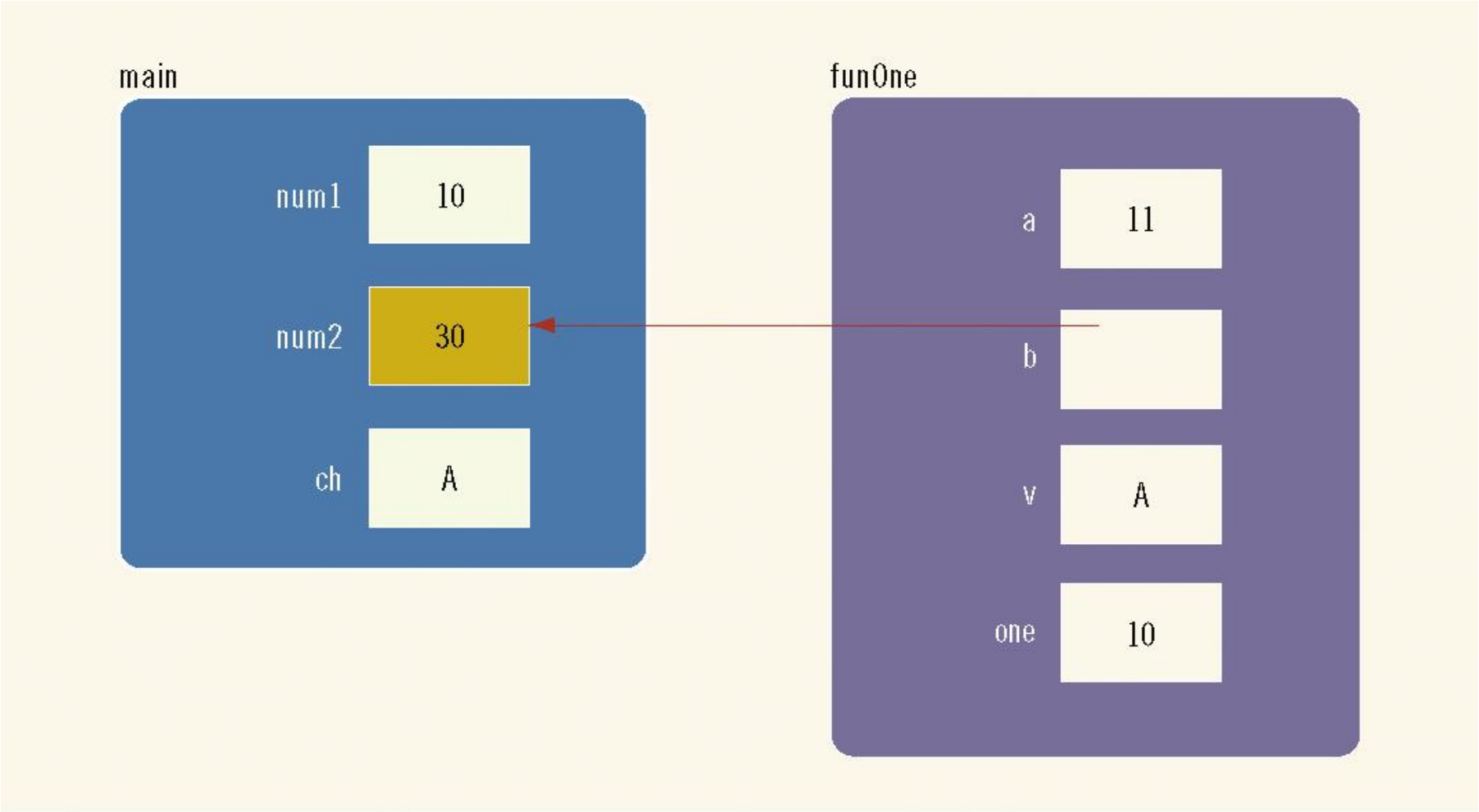

FIGURE 7-9 Values of the variables after the statement in Line 11 executes

```
v = 'B';                                                    //Line 12
```

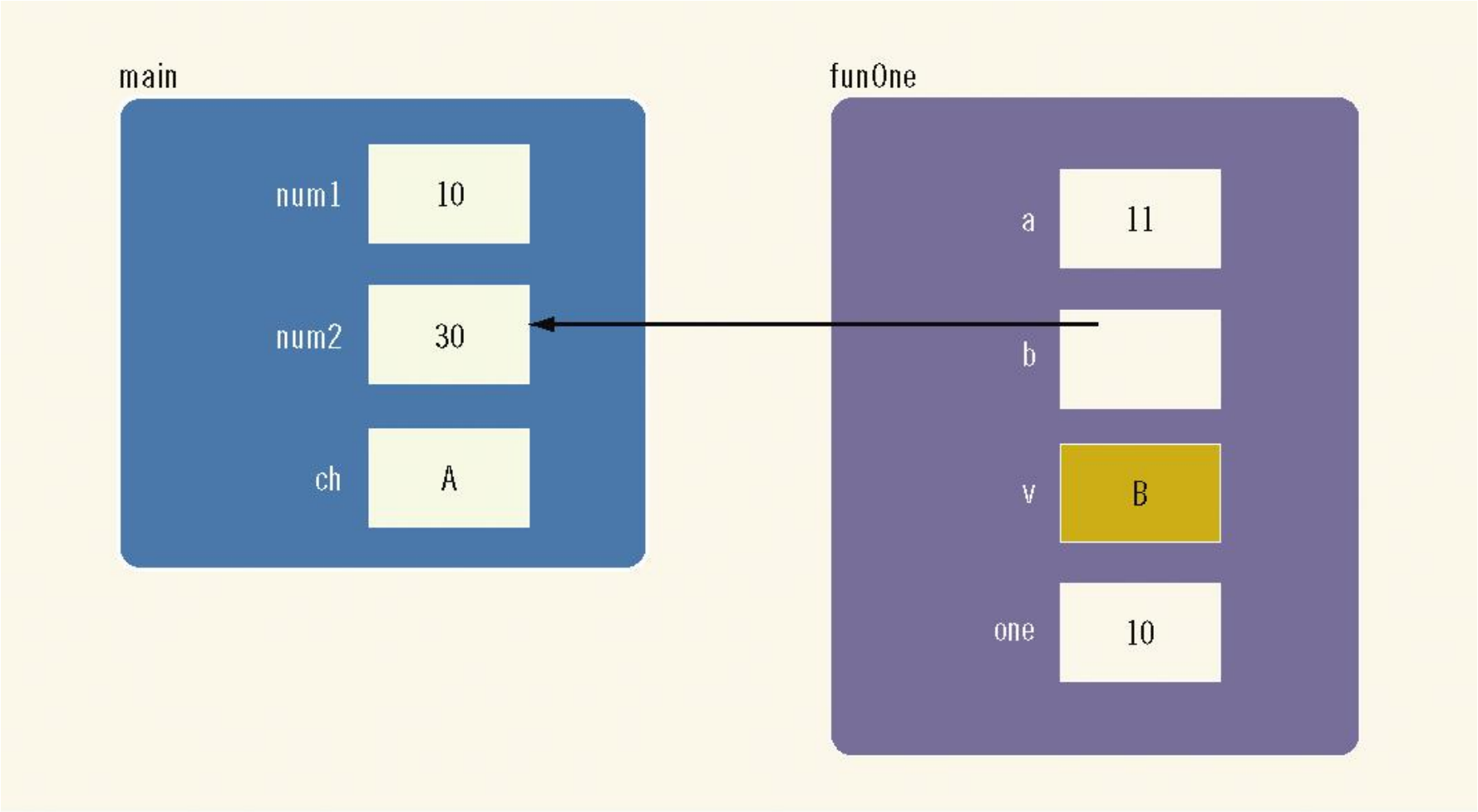

FIGURE 7-10 Values of the variables after the statement in Line 12 executes

```
cout << "Line 13: Inside funOne: a = " << a
     << ", b = " << b << ", v = " << v
     << ", and one = " << one << endl;          //Line 13
```

The statement in Line 13 produces the following output:

```
Line 13: Inside funOne: a = 11, b = 30, v = B, and one = 10
```

```
cout << "Line 6: After funOne: num1 = " << num1
     << ", num2 = " << num2 << ", and ch = "
     << ch << endl;                                  //Line 6
```

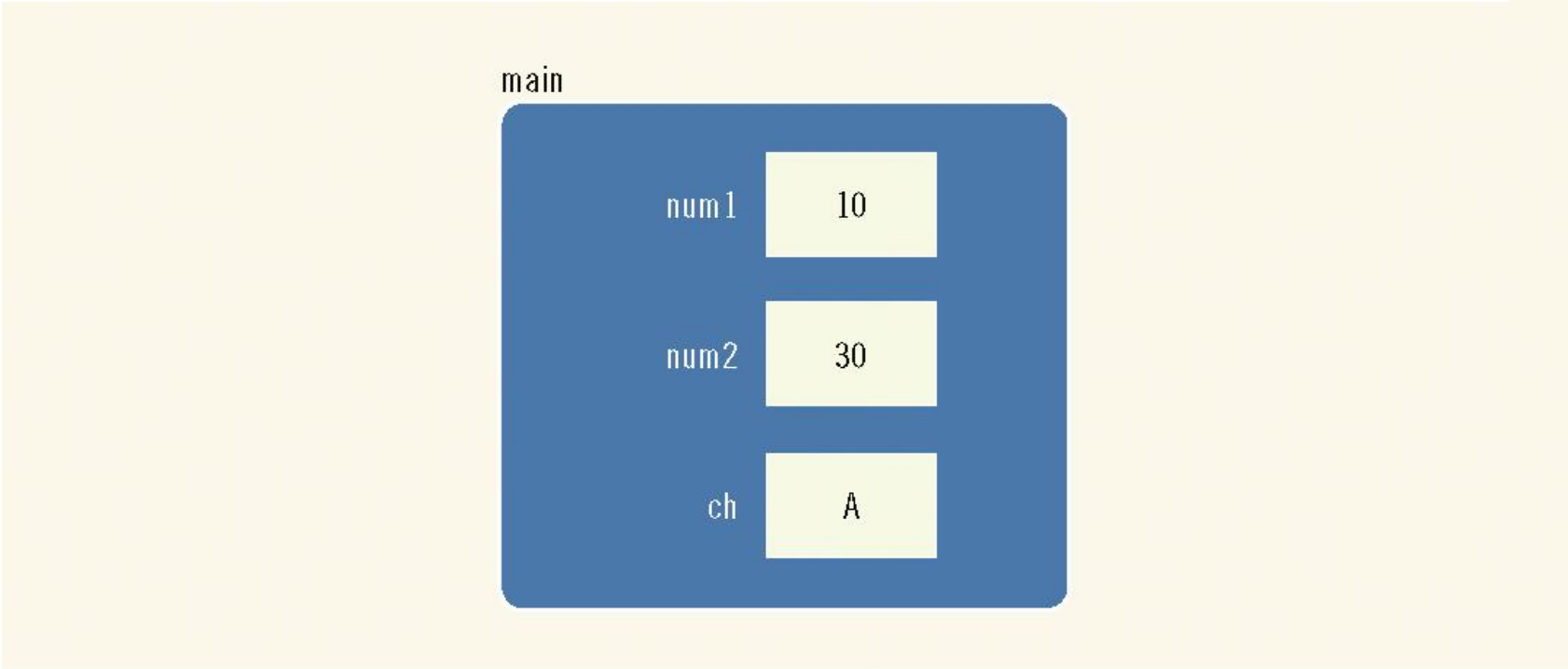

FIGURE 7-11 Values of the variables when control goes back to Line 6

Line 6 produces the following output:

```
Line 6: After funOne: num1 = 10, num2 = 30, and ch = A
```

```
x++;                                                    //Line 14
```

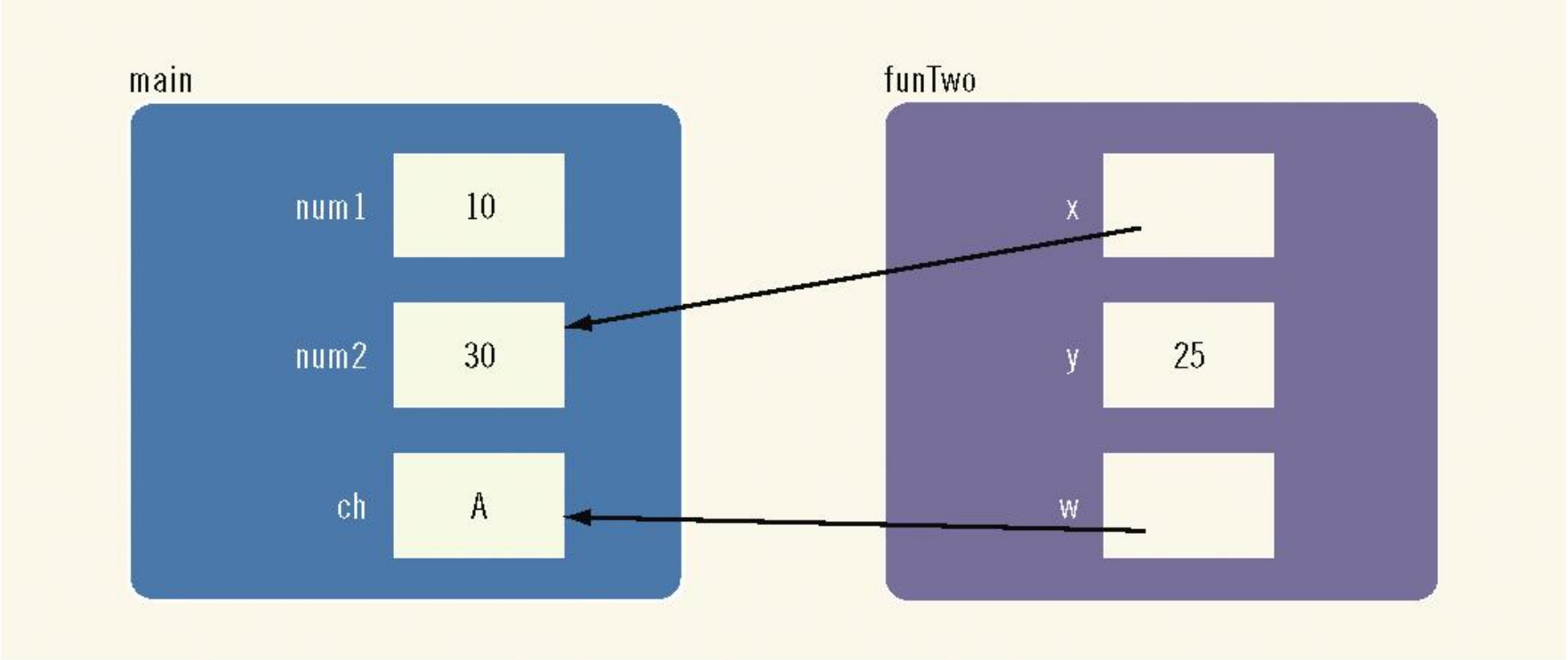

FIGURE 7-12 Values of the variables before the statement in Line 14 executes

```
x++;                                                    //Line 14
```

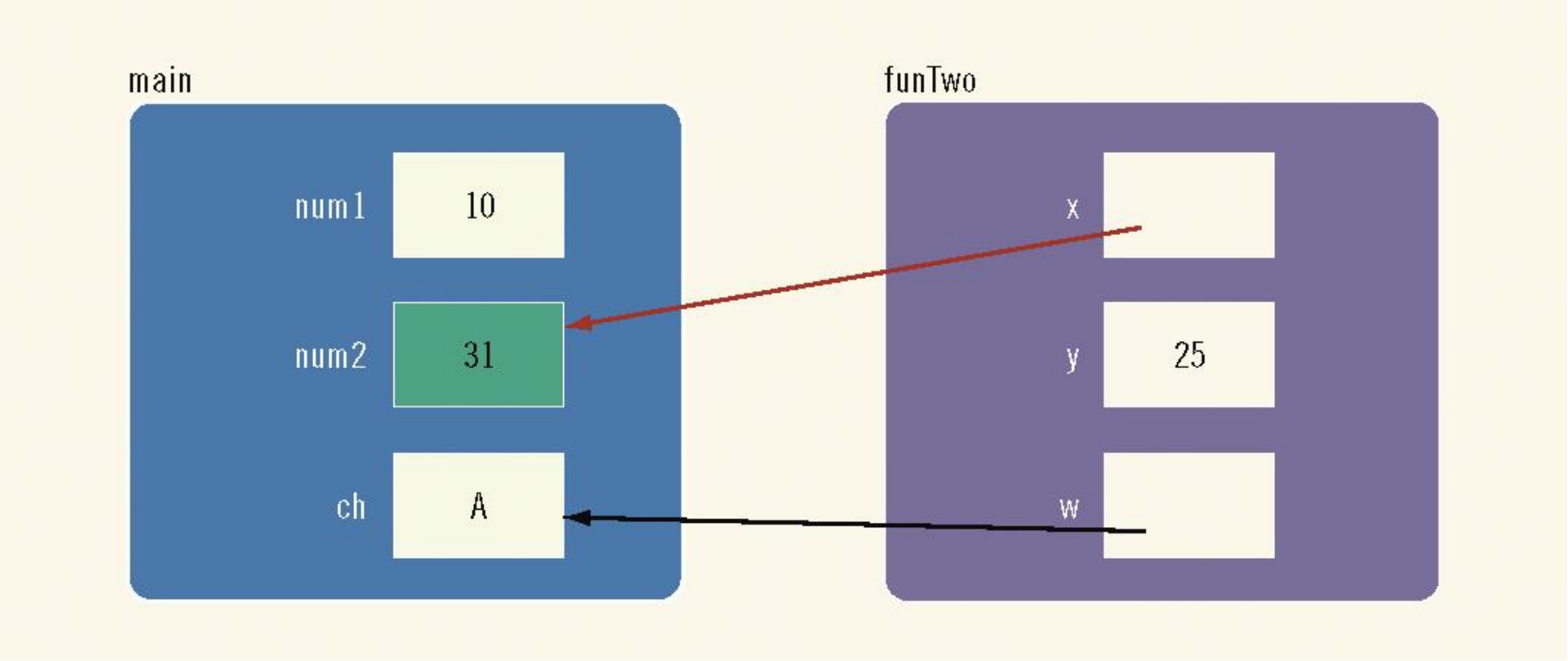

FIGURE 7-13 Values of the variables after the statement in Line 14 executes

```
y = y * 2;                                    //Line 15
```

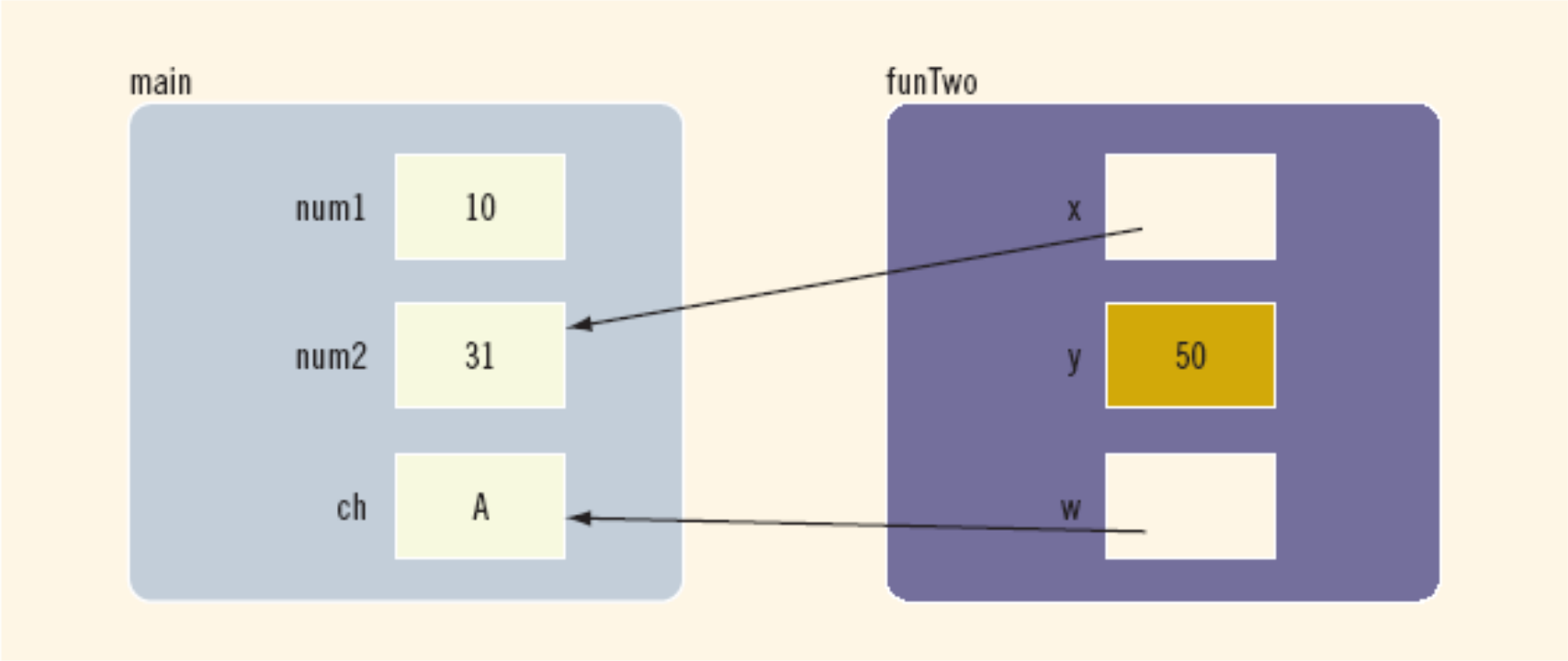

FIGURE 7-14 Values of the variables after the statement in Line 15 executes

```
w = 'G';                                                    //Line 16
```

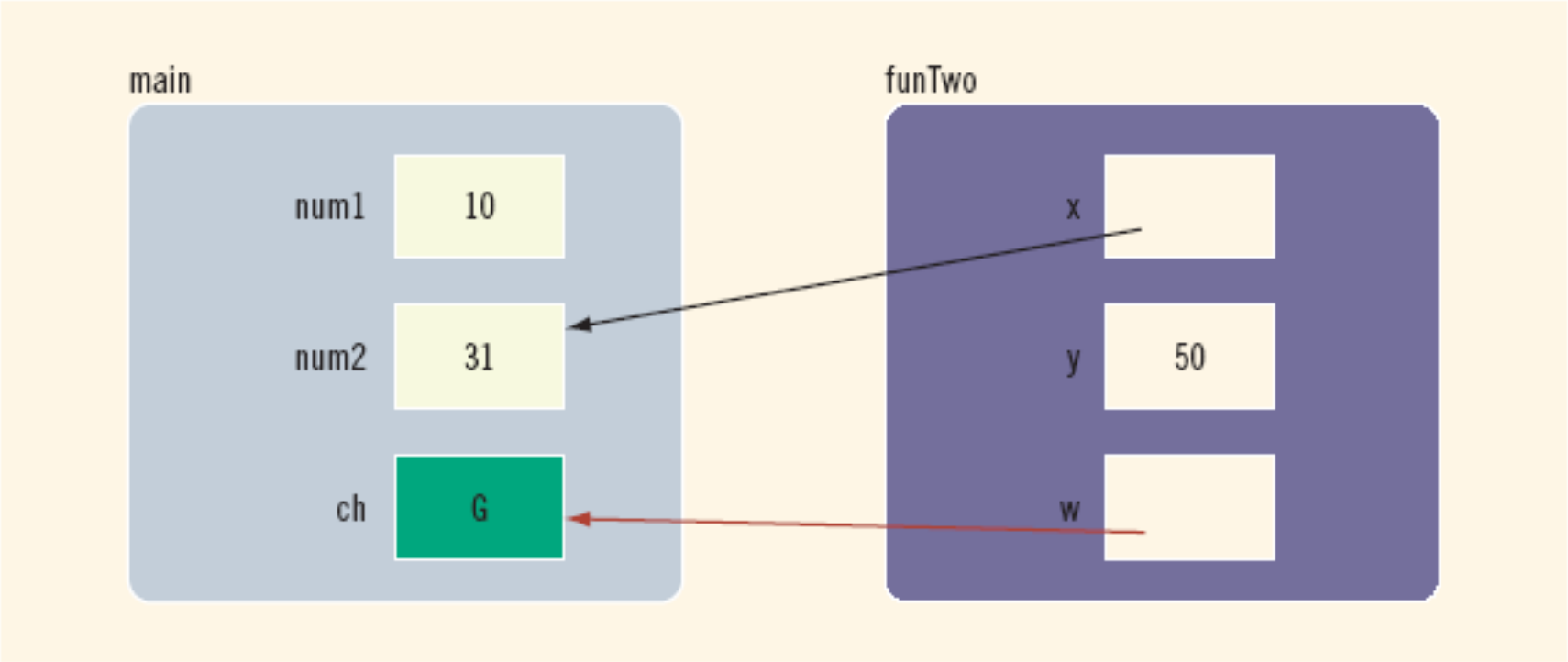

FIGURE 7-15 Values of the variables after the statement in Line 16 executes

```
cout << "Line 17: Inside funTwo: x = " << x
     << ", y = " << y << ", and w = " << w
     << endl;                                    //Line 17
```

Line 17 produces the following output:

```
Line 17: Inside funTwo: x = 31, y = 50, and w = G
```

```
cout << "Line 8: After funTwo: num1 = " << num1
     << ", num2 = " << num2 << ", and ch = "
     << ch << endl;                                  //Line 8
```

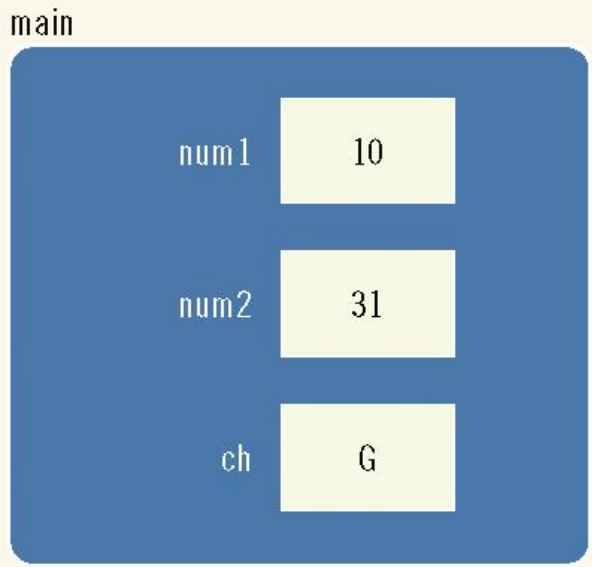

FIGURE 7-16 Values of the variables when control goes to Line 8

The statement in Line 8 produces the following output:

```
Line 8: After funTwo: num1 = 10, num2 = 31, and ch = G
```

Scope of an Identifier

- The scope of an identifier refers to where in the program an identifier is accessible
- Local identifier - identifiers declared within a function (or block)
- Global identifier – identifiers declared outside of every function definition
- C++ does not allow nested functions
  - The definition of one function cannot be included in the body of another function

In general, the following rules apply when an identifier is accessed:

1. Global identifiers (such as variables) are accessible by a function or a block if:
   a. The identifier is declared before the function definition (block),
   b. The function name is different from the identifier,
   c. All parameters of the function have names different than the name of the identifier, and
   d. All local identifiers (such as local variables) have names different than the name of the identifier.
2. **(Nested Block)** An identifier declared within a block is accessible:
   a. Only within the block from the point at which it is declared until the end of the block, and
   b. By those blocks that are nested within that block if the nested block does not have an identifier with the same name as that of the outside block (the block that encloses the nested block).
3. The scope of a function name is similar to the scope of an identifier declared outside any block. That is, the scope of a function name is the same as the scope of a global variable.

Global Variables

- Some compilers initialize global variables to default values
- The operator :: is called the scope resolution operator
- By using the scope resolution operator
  - A global variable declared before the definition of a function (block) can be accessed by the function (or block) even if the function (or block) has an identifier with the same name as the variable

Global Variables (continued)

- C++ provides a way to access a global variable declared after the definition of a function
- In this case, the function must not contain any identifier with the same name as the global variable

Side Effects of Global Variables

- Using global variables has side effects
- Any function that uses global variables
  - Is not independent
  - Usually cannot be used in more than one program

Side Effects of Global Variables (continued)

- If more than one function uses the same global variable and something goes wrong
  - It is difficult to find what went wrong and where
- Problems caused by global variables in one area of a program might be misunderstood as problems caused in another area

Static and Automatic Variables

- Automatic variable - memory is allocated at block entry and deallocated at block exit
- Static variable - memory remains allocated as long as the program executes
- Variables declared outside of any block are static variables
- By default variables declared within a block are automatic variables
- Declare a static variable within a block by using the reserved word static

Static and Automatic Variables (continued)

- The syntax for declaring a static variable is:

```
static dataType identifier;
```

- The statement

```
static int x;
```

  declares x to be a static variable of the type int

- Static variables declared within a block are local to the block
- Their scope is the same as any other local identifier of that block

Programming Example

- In this example
  - Using functions, we rewrite the program that determines the number of odds and evens from a given list of integers
  - This program was first written in Chapter 5
- The main algorithm remains the same:
  1. Initialize variables, zeros, odds, and evens to 0
  2. Read a number

Programming Example (continued)

3. If the number is even, increment the even count
   - If the number is also zero, increment the zero count; else increment the odd count
4. Repeat Steps 2 and 3 for each number in the list

Programming Example (continued)

- The program functions include:
  - Initialize: initialize the variables, such as zeros, odds, and evens
  - getNumber: get the number
  - classifyNumber: determine whether the number is odd or even (and whether it is also zero); this function also increments the appropriate count
  - printResults: print the results

```
void initialize(int& zeroCount, int& oddCount, int& evenCount)
{
    zeroCount = 0;
    oddCount = 0;
    evenCount = 0;
}
```

```
void getNumber(int& num)
{
    cin >> num;
}
```

```
void classifyNumber(int num, int& zeroCount, int& oddCount,
                    int& evenCount)
{
    switch (num % 2)
    {
    case 0:
        evenCount++;
        if (num == 0)
            zeroCount++;
        break;
    case 1:
    case -1:
        oddCount++;
    } //end switch
} //end classifyNumbers
```

```
void printResults(int zeroCount, int oddCount, int evenCount)
{
    cout << "There are " << evenCount << " evens, "
         << "which includes " << zeroCount << " zeros"
         << endl;

    cout << "The number of odd numbers is: " << oddCount
         << endl;
} //end printResults
```

Main Algorithm

1. Call function initialize to initialize variables
2. Prompt the user to enter 20 numbers
3. For each number in the list
   - Call function `getNumber` to read a number
   - Output the number
   - Call function `classifyNumber` to classify the number and increment the appropriate count
4. Call function `printResults` to print the final results

```
int main ()
{
        //Variable declaration
    int counter; //loop control variable
    int number;  //variable to store the new number
    int zeros;   //variable to store the number of zeros
    int odds;    //variable to store the number of odd integers
    int evens;   //variable to store the number of even integers

    initialize(zeros, odds, evens);                    //Step 1

    cout << "Please enter " << N << " integers."
         << endl;                                      //Step 2
    cout << "The numbers you entered are: "
         << endl;

    for (counter = 1; counter <= N; counter++)         //Step 3
    {
        getNumber(number);                             //Step 3a
        cout << number << " ";                         //Step 3b
        classifyNumber(number, zeros, odds, evens); //Step 3c
    } // end for loop

    cout << endl;

    printResults(zeros, odds, evens);                  //Step 4

    return 0;
}
```

Summary

- Void function: a function that does not have a data type
- A `return` statement without any value can be used in a void function to exit function early
- The heading of a `void` function starts with the word void
- To call a `void` function, you use the function name together with the actual parameters in a stand-alone statement

Summary (continued)

- Two types of formal parameters: value parameters and reference parameters
- A value parameter receives a copy of its corresponding actual parameter
- A reference parameter receives the address (memory location) of its corresponding actual parameter

Summary (continued)

- If a formal parameter needs to change the value of an actual parameter, in the function heading you must declare this formal parameter as a reference parameter
- Variables declared within a function (or block) are called local variables
- Variables declared outside of every function definition (and block) are called global variables

Summary (continued)

- An automatic variable is a variable for which memory is allocated on function (or block) entry and deallocated on function (or block) exit
- A static variable is a variable for which memory remains allocated throughout the execution of the program